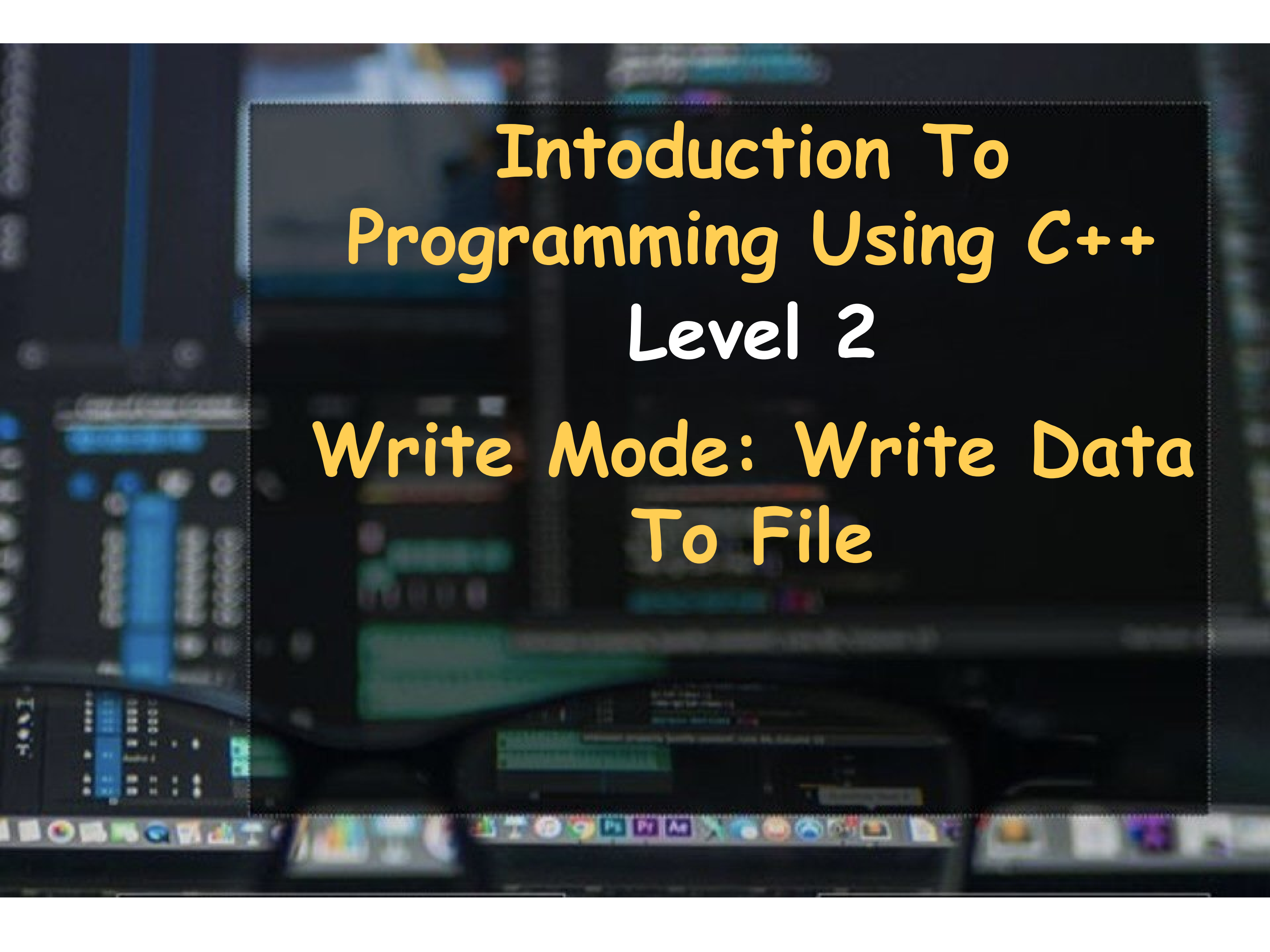


The background of the slide is a blurred image of a computer monitor. The monitor shows a code editor with syntax-highlighted text in various colors (blue, green, yellow) on a dark background. At the bottom of the monitor, a Windows taskbar is visible, featuring several application icons including a web browser, a file explorer, and various development tools. The overall scene is dimly lit, focusing attention on the text overlay.

Introduction To Programming Using C++ Level 2

Write Mode: Write Data To File



Introduction To Programming Using C++

Level 2

Write Mode: Write Data To File

Definition

Write Data To File

 Writing data to a text file in C++ means opening a file in write mode and storing data inside it.

 The ``<fstream>` library provides the classes and functions needed to work with files, including opening, writing, reading, and closing files.

Explanation

``<fstream>``

 We include the ``<fstream>`` library to work with files.

```
#include <fstream>
```

``fstream``

 ``fstream`` is a stream class that can be used for both reading from and writing to files.

 We create an object such as:

```
fstream MyFile;
```

Opening the File

 We use ``open()`` to open the file.

```
MyFile.open("MyFile.txt", ios::out);
```

 ``ios::out`` opens the file for output (writing).

``is_open()``

 ``is_open()`` checks whether the file was successfully opened.

```
if (MyFile.is_open())
```

Writing Data

 We use the insertion operator ``<<`` to write data into the file.

 It works similarly to ``cout``, but the output is written to the file instead of the screen.

```
MyFile << "Hi, this is the first line\n";
```

Closing the File

 We use ``close()`` after finishing the file operations

Procedures

1. Include the `<fstream>` library.
2. Create an `fstream` object.
3. Open the file using `open()` with `ios::out`.
4. Check whether the file opened successfully using `is_open()`.
5. Write data using the `<<` operator.
6. Close the file using `close()`.

💡 Example

```
int main()
{
    fstream MyFile;
    MyFile.open("MyFile.txt", ios::out);
    if (MyFile.is_open())
    {
        MyFile << "Hi, this is the first line\n";
        MyFile << "Hi, this is the second line\n";
        MyFile << "Hi, this is the third line\n";
        MyFile.close();
    }
    return 0;
}
```

➡ Result

A file named `MyFile.txt` is created or opened, and it contains:

Hi, this is the first line

Hi, this is the second line

Hi, this is the third line

Note

◆ You can specify a complete file path, for example:

`E:/MyDataFolder/CppProjects/Lesson-52.txt`

◆ If you provide only the file name, such as:

`MyFile.txt`

the file is opened relative to the program's current working directory.

◆ `<<` is the correct operator for writing data to a file.

◆ `>>` is normally used for extracting data from a stream, such as reading data from a file.

⊘ Warning

- ◆ Be careful when using ``ios::out`` with an existing file because opening for output with truncation can erase the previous contents.
- ◆ If you want to preserve existing content and add new data at the end, use ``ios::app`` instead.
- ◆ Always check whether the file opened successfully before writing to it.



Comparison

`ios::out`

- ✓ Opens a file for output/writing.
- ✓ Commonly used when replacing the existing file contents.
- ✓ Existing contents can be truncated when the file is opened.

`ios::app`

- ✓ Opens a file for output.
- ✓ Preserves existing contents.
- ✓ New data is written at the end of the file.

✦ Summary

- ✓ `<fstream>` provides tools for working with files.
- ✓ `fstream` creates a file stream object.
- ✓ `ios::out` opens a file for writing.
- ✓ `is_open()` checks whether the file opened successfully.
- ✓ `<<` writes data into the file.
- ✓ `close()` closes the file after finishing.
- ✓ Be careful with `ios::out` because existing file contents may be erased.

Thank you 😊